MSCPHUB MSCP Disk Server for OpenVMS Nodes

Programmer’s Guide

MSCPHUB-PG-0116

MSCPHUB Version X01-16 *** 22-OCT-2026

Contents

Preface **3**
Intended Audience . . . 3
Document Structure . . . 4
Conventions . . . 5

Chapter 1. Concept **6**
1.1 Why the Server Exists . . . 6
1.2 What the Server Is . . . 7
1.3 Decisions and Why . . . 8
1.4 What Is Deliberately Not Done . . . 9

Chapter 2. Architecture **10**
2.1 The Layers . . . 10
2.2 The Event Loop . . . 11
2.3 The Path of a READ . . . 12

Chapter 3. The Modules **13**
3.1 mscphub.c . . . 13
3.2 mh_lan.c . . . 14
3.3 mh_cc.c . . . 15
3.4 mh_tr.c . . . 16
3.5 mh_ppd.c . . . 17
3.6 mh_scs.c . . . 18
3.7 mh_blk.c . . . 19
3.8 mh_mscp.c . . . 20
3.9 mh_bk.c . . . 21
3.10 mh_utl.c . . . 22
3.11 mh_hlp.c . . . 23

Chapter 4. Data Structures **24**

Chapter 5. Conventions **25**
5.1 Naming . . . 26
5.2 Formatting . . . 27
5.3 The Module Header . . . 28
5.4 IDENT and REV . . . 29
5.5 The Calling Convention . . . 30
5.6 Diagnostics . . . 31
5.7 Strings and Lengths . . . 32
5.8 The Build Files and the Shell . . . 33
5.9 The Documentation . . . 34

Chapter 6. Making a Change **35**
6.1 Adding a Qualifier . . . 35
6.2 Adding a Message . . . 36
6.3 Adding an MSCP Command . . . 37

6.4 Releasing a Version 38

Chapter 7. The Build System **39**
7.1 Layout 39
7.2 Dependencies 40
7.3 Options 41
7.4 The Checks of the Configuration 42
7.5 The PDF of the Manuals 43
7.6 The Kit 44

Chapter 8. Testing **45**
8.1 What Is There 45
8.2 Rules 46

Appendix A. Known Limitations **47**

Preface

Intended Audience

This manual is for the programmer who is to alter the server: to add a qualifier, a message or an MSCP command, to correct a defect, or to satisfy himself that a change is safe. It assumes C, POSIX and a reading acquaintance with CMake. It does not assume OpenVMS, although the reader who knows it will recognize why much of the design looks as it does.

The *MSCPHUB User's Guide* describes what the server does. This manual describes how it is made, and neither repeats the other.

Document Structure

Part	Contents
Chapter 1	Why the server exists, and the decisions it rests upon
Chapter 2	Architecture: the layers, the event loop, the path of a command
Chapter 3	The modules, one by one
Chapter 4	The data structures
Chapter 5	The conventions the sources hold to, and why
Chapter 6	How to make the changes that come up most often
Chapter 7	The build system and the kit
Chapter 8	The tests and what they guard
Appendix A	Known limitations

Conventions

Line numbers are not quoted: they go stale. A routine is named instead, and every routine carries a `FUNCTIONAL DESCRIPTION` block at its head that says what it does, what it takes and what it returns. Where this manual and that block disagree, the block is right and this manual is to be corrected.

Chapter 1. Concept

1.1 Why the Server Exists

The VMScluster of Digital Equipment Corporation was the first cluster that worked: every node sees every disk, a disk is served by whichever node or controller holds it, and the cluster carries on when a member leaves. The machines that ran it are leaving the machine rooms, and the controllers that served its disks - the HSC, the HSJ, the HSD - left long ago. The systems themselves, and the engineering culture they were made with, should not go the same way.

MSCPHUB is one part of the work to keep that heritage alive and in use. It takes the place of the disk controller: OpenVMS nodes - real or emulated, Alpha, Integrity or x86 - keep their disks on the storage of a Linux computer, and nothing in OpenVMS is changed for it.

The second purpose is the culture itself. The sources are written the way the engineers of DEC wrote theirs: the facility and its condition values, messages in the OpenVMS form, a HELP library, qualifiers, IDENTs and a modification history in every module, a `FUNCTIONAL DESCRIPTION` over every routine, manuals laid out like the manuals of OpenVMS. This is deliberate. It is the way of working that made those systems last for forty years - every decision written down, every condition reported, nothing left for the operator to guess - and a way of working is kept only by being used. A programmer who alters the server is asked to keep it so (Chapter 5).

1.2 What the Server Is

The server is an SCS node with one system application, the disk server `MSCP$DISK`, and without a connection manager. It speaks NISCA - the protocol of PEDRIVER - on the Ethernet, so that an OpenVMS node finds it on the LAN, opens a channel and a virtual circuit to it, connects its disk class driver to it, and sends it MSCP commands. It is never a member of the VMScluster and never votes.

1.3 Decisions and Why

No membership in the cluster. A member would have to run the connection manager, take part in state transitions and the quorum, and could hang the cluster by a fault of its own. A controller that only serves disks needs none of it. The server tells OpenVMS that it is not an OpenVMS system, and OpenVMS then treats it as it treats an HSC.

One process, one thread. The protocol, from the frame up to MSCP, runs in one thread on an EPOLL loop. Nothing is locked, because nothing else touches the state. This is a real constraint on what may be added: anything that blocks the loop stops every node. The disk I/O is therefore never done in the loop; it goes to io_uring, and its completions come back as an event of the loop.

The answer to a WRITE means that the data is on the medium. OpenVMS never asks a disk to flush a cache of its own. A server that answers a WRITE from the page cache would lose data that OpenVMS believes is written. The default is therefore a durable write for every WRITE; `/UNSAFE` exists for tests and says so when it starts.

Never send what can crash a node. A malformed answer of a disk server can make an OpenVMS node crash. The server checks its own answers before they leave and holds back one that would be wrong (`MSCPERR`); the strictness of the block transfers comes from the same rule.

Silence before speech. The server listens before it sends its first frame, and stops if its name or system ID is already on the LAN: a clash of identity with a node of the cluster can make that node leave the cluster.

Condition values, not error codes. Every routine returns an OpenVMS condition value; the low bit is the success indicator. A failure is told apart by its identification, never by its text.

Someone else’s library. Logging, messages, the parsing of the command line, counted strings and the FAO engine come from the StarLet package; the help library comes from the HELP package. The server carries no general-purpose code of its own.

1.4 What Is Deliberately Not Done

- **No membership, no votes, no connection manager** (Section 1.3).
- **No cache of data in the server.** The data of a READ comes from the Linux disk, and the page cache of Linux is not relied upon for a WRITE. The one exception is the cache of the sectors last written on a disk of 4096-octet sectors, which only spares the reads of a read-modify-write.
- **No threads.** The backend is io_uring, not a pool of threads.
- **No fallback for old kernels.** io_uring and the other interfaces of the kernels of the last five years are required; there is no second path for an older kernel.

Chapter 2. Architecture

2.1 The Layers

Bottom up:

Module	Layer	What it does
`mh_lan.c`	LAN	The AF_PACKET socket: frames of type 60-07 in and out, the multicast of the group
`mh_cc.c`	Channel control	HELLO, the table of the nodes seen, the channel to each node
`mh_tr.c`	Transport	The virtual circuit: sequence numbers, acknowledgement, window, retransmission
`mh_ppd.c`	Path	The START/STACK/ACK handshake of SCS, HOSTSHUT
`mh_scs.c`	SCS	Connections, credits, the directory service `SCS$DIRECTORY`
`mh_blk.c`	Block transfers	SEND_DATA for a READ, REQUEST_DATA for a WRITE
`mh_mscp.c`	MSCP server	The units, the hosts, the commands and their order
`mh_bk.c`	Backend	The disk I/O of the units, by io_uring
`mh_utl.c`	Services	The message catalogue, the OpenVMS time, conversions
`mh_hlp.c`	Help	`/HELP`, out of the help library
`mscphub.c`	Head module	The parameters, the start-up checks, the event loop

The definitions they share are in `include/mh_def.h`; the formats of NISCA and of MSCP are in `include/nisca.h` and `include/mscp.h`.

2.2 The Event Loop

`s_mh$loop` in `mscphub.c` waits on EPOLL for four sources:

Source	Event
The LAN socket	Frames: drained, each given to `mh$cc_input`; then the acknowledgements owed go (`mh$tr_flush`)
A TIMERFD of 20 ms	The tick of the protocol: `mh$cc_tick` (HELLO, timers of the channels and circuits), `mh$bk_tick` (stuck requests)
The EVENTFD of io_uring	Disk completions: `mh$bk_reap`
A SIGNALFD	SIGINT and SIGTERM stop the server; SIGUSR1 writes the state

A routine of any layer that returns a condition of severity F stops the loop; the server then says goodbye to the nodes (`mh$cc_shutdown`), writes its counters and exits.

2.3 The Path of a READ

1. A frame comes in. `mh$cc_input` checks the DX header and the group, and hands the TR packet of an open channel to the transport.
2. The transport (`mh_tr.c`) takes the packet in order, frees what its acknowledgement covers, and gives the message to the path and to SCS.
3. SCS (`mh_scs.c`) finds the connection, takes the credit, and gives the application message to the MSCP server.
4. The MSCP server (`mh_mscp.c`) checks the command - length, opcode, flags, modifier, unit - and makes it a command in progress (`MH$HRB`). It keeps the order of the unit: a sequential command waits for the unit to be idle; READ, WRITE, ERASE and COMPARE run side by side.
5. The backend (`mh_bk.c`) reads the disk into a data buffer by io_uring. The loop goes on serving.
6. The completion comes on the EVENTFD; `mh$bk_reap` gives the request back to the MSCP server.
7. The block transfer (`mh_blk.c`) sends the data to the buffer of the node in chunks, as the window of the transport opens. The node confirms the last chunk.
8. The MSCP server sends the end message. The transport carries it; the acknowledgement of the node's packets rides on it.

A WRITE goes the other way: the server asks for the data (REQUEST_DATA), the node's PEDRIVER sends it in chunks, the backend writes it durably, and only then does the end message go.

Chapter 3. The Modules

3.1 mscphub.c

The head module. It holds the table of the qualifiers (`s_opts`), checks the configuration (`s_mh$check_cfg`), opens the layers in order and runs the loop. Its `__IDENT__` is the version of the product: CMake reads it from here, `STARTUP` and `UPTIME` report it. `s_mh$stats` writes the messages of `SIGUSR1`.

3.2 mh_lan.c

The socket is `SOCK_RAW` of `AF_PACKET`, bound to the interface and to type 60-07. The Ethernet header and the length word of the DEC padding are built here; the frame is padded to the Ethernet minimum. Frames the server has sent itself come back to such a socket and are dropped and counted. `a_port->xmit`, when set, takes a frame in place of the socket: the tests use it.

3.3 mh_cc.c

The DX header, the CC messages, HELLO, the table of the nodes seen on the LAN, and the state machine of the channel to each node. Before its first HELLO the server is silent for `/LISTEN` seconds and learns the nodes; a node that uses its system ID or its name stops it (`IDCLASH`, `NAMECLASH`), and the check stays on for the life of the server. A CC message with a wrong password hash is counted and does not move the channel (`AUTHFAIL`). With `/PASSIVE` the state machine does not run.

3.4 mh_tr.c

The transport of the virtual circuit: sequence numbers in longwords, a cumulative acknowledgement, a window of at most 16 packets, a cache of the packets that come out of order, and retransmission - 500 ms, doubled up to 3 s; 30 timeouts in a row fail the circuit. An acknowledgement owed goes on the next packet of the server, or alone as `/ACK` says. The window is filled from the queue of messages first, then with the chunks of the block transfer at the head of the circuit, which are made only when there is room.

3.5 mh_ppd.c

The path of SCS to a node over its circuit: the server answers the START of the node with STACK, takes the system block of the node out of the START, and opens its transport. HOSTSHUT closes it.

3.6 mh_scs.c

The acceptor half of SCS: the server accepts connections and never asks for one. Two listeners: `SCS$DIRECTORY` and, when disks are served, `MSCP$DISK`. Credits are spent on every application message sent and given back on the next. The directory answers lookups: `MSCP$DISK` is present when disks are served, the other well-known names are answered “not present here”.

3.7 mh_blk.c

The block transfers. SEND_DATA (READ): the chunks go as the window opens, one transfer at a time per circuit; the end message goes only after the node confirms the last chunk. REQUEST_DATA (WRITE, COMPARE): one request for all the data; the chunks come in. A chunk never goes past the byte count of its command; a chunk that comes for nothing of the server’s is dropped and reported once (`BLKERR`).

3.8 mh_mscp.c

The MSCP server: units, hosts (one per connection of a disk class driver), the check of a command, the order of the commands on a unit, and the commands. Immediate commands (ABORT, GET COMMAND STATUS, GET UNIT STATUS, SET CONTROLLER CHARACTERISTICS and the like) are answered at once; the others become an `MH$HRB`. Data buffers come from a pool of 16 MB; a command that finds it empty waits. An end message that would crash the node is held back (`MSCPERR`) and the connection ended. The counters of `UNITSTATS`, `BKSTATS` and `LATSTATS` are kept here.

3.9 mh_bk.c

The backend. The MSCP server hands it a request (`MH$BKREQ`); it goes to io_uring as one step or a few. A device of sectors larger than 512 octets takes a request that is not on whole sectors through a bounce buffer: a READ reads the whole sectors and copies the middle out, a WRITE reads the first and the last sector (or finds them in the cache of the 128 sectors last written), puts its data between them and writes them all. Every WRITE of such a device holds its range of sectors; one that meets a range held waits, in order. A WRITE is made durable as `/SYNC` says. ERASE writes zeros to a file up to 1 MB and uses `fallocate` beyond. A request in the backend longer than 15 seconds is reported (`IOSTUCK`) and cancelled.

A file is opened twice: once with `O_DIRECT` for the I/O of whole blocks, once through the page cache for the rare count that is not a multiple of 512. A file system that refuses `O_DIRECT` leaves the file on the page cache (`NODIRECT`).

3.10 mh_utl.c

The message catalogue (`s_msgtab`) and its registration, the OpenVMS time and its zone, conversions of hexadecimal and MAC address notations.

3.11 mh_hlp.c

`/HELP`, answered out of the help library by the HELP package. The library is looked for in `MSCPHUB_HELPLIB`, in the installed tree, beside the image, and where the build put it. Built without the HELP package, the server answers `/HELP` by a short summary.

Chapter 4. Data Structures

All are declared in `include/mh_def.h`.

Structure	What it is
`MH$CFG`	The configuration: the values of the qualifiers
`MH$PORT`	The LAN port and everything below it: the socket, the table of the nodes, the counters
`MH$NODE`	A node seen on the LAN: its identity and the state of the channel to it
`MH$VC`	The virtual circuit to a node: the numbers, the window, the queues, the connections
`MH$TRPKT`	A packet in the transmit queue of a circuit
`MH$CDT`	An SCS connection: its state, its credits, its queue
`MH$APPLMSG`	An application message waiting for a credit
`MH$UNIT`	A unit of the MSCP server: its name, size, state and counters
`MH$HRB`	A command in progress
`MH$BKDEV`	The backend side of a unit: the descriptors, the sector, the cache of written sectors
`MH$BKREQ`	A request to the backend
`MH$LAT`	The time counters of one operation of a unit

The ownership is simple, because there is one thread: a structure belongs to the layer that declares its state, and a lower layer never frees what an upper one holds. A command in progress (`MH$HRB`) outlives its connection when the disk still has a request of it; it is freed when the backend gives the request back.

Chapter 5. Conventions

The sources hold to the OpenVMS style throughout. This is not decoration: a reader who knows the convention can tell the lifetime and the origin of a name at sight, and the next programmer reads the code the way the last one wrote it. A change that does not hold to these rules is not finished.

5.1 Naming

Form	Meaning
`l_name`	Local variable
`s_name`	Static: a variable, or a routine not exported from its module
`a_name`	Parameter of a routine
`g_name`	Global variable (the server has none)
`m_name`	Bit mask
`v_name`	Bit field in a structure
`mh$module_name`	Exported routine: the facility, the dollar, the module, the name
`s_mh$module_name`	Static routine
`MH$K_NAME`	Constant
`MH$_NAME`	Condition value
`MH$NAME`	Type of a structure
`$NAME`	Macro

The dollar separates the words of a name, as in OpenVMS: `MH$K_BK_CACHE`, `$MHMSG`. A structure is declared with the tag carrying the suffix `_t` and the type name carrying the dollar; the brace stands at column zero and the members are indented:

```
typedef struct mh_lat_t {
    uint64_t    n;          /* Commands                */
    uint64_t    sum, max;       /* From the command to the END          */
} MH$LAT;
```

Ordered constants are an `enum`. Integers carry their size in their type: `uint32_t`, `uint16_t`; values on the wire are put and taken by `nisca$put16`, `nisca$get32` and their kin, never by casting a pointer.

5.2 Formatting

- **Whitesmiths.** The brace of a block stands on its own line at the indentation of the body it opens. The braces of a routine body stand at column zero.
- **Tabs only**, of width eight. No spaces for indentation.
- **Text to column 192**, not 80.
- **Declarations at the head of a routine**, without indentation, grouped by type.
- **A single statement under `if` takes no braces.**
- **A compound condition is parenthesized part by part**, so that nothing depends on the precedence of operators: `if ( (a_len > MH$K_MAX) || !a_buf )`.
- **A loop counter is declared in the `for`**: `for ( int i = 0; i < n; i++ )`.

A routine is laid out so:

```
/*
**++
**  FUNCTIONAL DESCRIPTION:
**
**  Close the port.  The memberships go with the socket.
**
**  FORMAL PARAMETERS:
**
**  a_port      The port context
**
**  RETURN VALUE:
**  STS$K_SUCCESS.
**--
*/
int mh$lan_close    (
        MH$PORT *   a_port
            )
{
    if ( a_port->sd >= 0 )
        close(a_port->sd);

    a_port->sd = -1;

    return  STS$K_SUCCESS;
}
```

The block over a routine says what it does, its `FORMAL PARAMETERS`, its `IMPLICIT INPUTS/OUTPUTS` where it has them, and its `RETURN VALUE`. Comments are detailed where the code is not obvious and absent where it is: half a screen over a constant or a routine of one line is noise.

5.3 The Module Header

Every source file begins with its identification and its history:

```
#define __MODULE__  "MHLAN"
#define __IDENT__   "X01-10"
#define __REV__     "1.10.0"

/*
**++
**
**  FACILITY:   MSCPHUB - MSCP disk server for OpenVMS nodes, over NISCA
**
**  MODULE: mh_lan.c
**
**  ABSTRACT:   ...
**
**  DESCRIPTION: ...
**
**  AUTHOR: StarLet Squad and Ruslan R. Laishev (AKA: BadAss SysMan)
**
**  CREATION DATE:  17-OCT-2026
**
**  MODIFICATION HISTORY:
**
**      20-OCT-2026 RRL X01-10 : Initial public version.
**
**--
*/
```

The author is always “StarLet Squad and Ruslan R. Laishev (AKA: BadAss SysMan)”, and every line of the history carries `RRL` between the date and the IDENT.

5.4 IDENT and REV

`__IDENT__` is the version in the OpenVMS manner: `X`*aa*`-`*bb*, and `X`*aa*`-`*bb*`ECO`*cc* for a small correction of a fault, which does not raise *bb*. `__REV__` is the same version in the Unix manner, derived from it part for part: `X01-15` gives `1.15.0`, `X01-11ECO1` gives `1.11.1`. The two are kept together with care; CMake refuses to configure a tree in which a module's `__REV__` does not follow its `__IDENT__`.

A module carries the IDENT of the last version that changed it. The IDENT of the product is that of the head module, `mscphub.c`, and nowhere else is it taken from.

5.5 The Calling Convention

A routine returns a condition value and delivers its result through an output parameter. A condition is tested on its low bit:

```
if ( !(1 & mh$lan_open(&s_port, $ASCPTR(&s_cfg.ifname))) )
    return  1;
```

`$MHMSG` returns the condition it was given, which is what lets a message and a return fit into one statement.

5.6 Diagnostics

- **What the operator is to read** goes through `$MHMSG`, a condition of the catalogue in `mh_utl.c`, in the OpenVMS form: `%MSCPHUB-F-IDCLASH, ...`. The text is in the catalogue, not at the point of the call.
- **What the programmer is to read** goes through `$LOG_FAO` with an explicit `STS$K_*` severity.
- Nothing calls `printf` in any of its forms. Output goes through the FAO engine of StarLet, in the OpenVMS directives: `!AZ`, `!UL`, `!SL`, `!UQ`, `!XW`, `!%S` and the rest.
- **The failure of a C RTL routine is reported where it happens**, with the name of the routine and `errno`:

```
if ( 0 > (a_port->sd = socket(AF_PACKET, SOCK_RAW | SOCK_NONBLOCK | SOCK_CLOEXEC, htons(NISCA$K_ETHERTY
    return  $MHMSG(MH$_SYSERR, "socket(AF_PACKET)", errno, strerror(errno));
```

Returning a condition upward and leaving the account of the run-time library behind loses the only information that would have identified the failure.

5.7 Strings and Lengths

A string is carried with its length: the counted strings of StarLet (`ASC`, `$ASCLEN`, `$ASCPTR`). A length once known is not computed again; `strlen` over the same string twice is a fault of style.

5.8 The Build Files and the Shell

- No dollar sign in an identifier of a CMake file: CMake would take it for the start of a reference. The constants of the build are named `MH_`*NAME*.
- No dollar sign in the name of an environment variable: `MSCPHUB_HELPLIB`, not `MSCPHUB$HELPLIB`.

5.9 The Documentation

- Every manual exists in English and in Russian, and the two say the same things.
- The English is plain: short sentences, common words. Many readers read it as a second language.
- Every message has a topic in both help libraries: what it means, and what to do about it.
- The manuals are laid out like the manuals of OpenVMS: a title page with the order number, a preface, numbered chapters, a command dictionary, a chapter of messages, a chapter of troubleshooting.

Chapter 6. Making a Change

6.1 Adding a Qualifier

1. Add the field to `MH$CFG` in `mh_def.h`.
2. Add a line to `s_opts` in `mscphub.c`: the name in lowercase, the field, the size, the type.
3. Check its value in `s_mh$check_cfg`; a wrong one is `MH$_IVPARAM`, saying what is expected.
4. Document it: a topic `## /NAME -- ...` in both help libraries, a paragraph in the manual page, an entry in the command dictionary of both User's Guides.

The test `docs` fails until the topics and the paragraph are there: it takes every name of `s_opts` and looks for them.

6.2 Adding a Message

1. Add the message number to the `enum` of `mh_def.h` and the condition `MH$_`*NAME* with its severity.
2. Add the record to `s_msgtab` in `mh_utl.c`: the identification, a comma, the text with FAO directives.
3. Document it: a topic *###NAME* in both help libraries, an entry in Chapter 6 of both User's Guides.

The test `docs` fails until the topics are there.

6.3 Adding an MSCP Command

The table of the commands, `s_opdef` in `mh_mscp.c`, gives each opcode the length of the command it takes (0: not taken), the length of its end message, the modifiers it may carry, and its name. Whether a command is immediate, sequential or runs side by side is decided by its opcode where the command is taken in. A command that moves data has steps in `mh_mscp.c` and a request type in `mh_bk.c`. Every new end message must pass the check that holds back an answer a node would crash on; extend the check if the new message has fields it does not know.

6.4 Releasing a Version

1. Raise `__IDENT__` and `__REV__` of the head module, and of every module changed.
2. Add a line to the modification history of every file changed.
3. Update the title blocks of the manuals: the order number, the version, the date, and what the edition supersedes.
4. `ctest` must pass.
5. `cmake --build build --target dist` makes `MSCPHUB-`*ident*`.tar.gz` of the committed sources.

Chapter 7. The Build System

7.1 Layout

Directory	Contents
`src/`	The modules
`include/`	`mh_def.h`, `nisca.h`, `mscp.h`
`doc/`	The manuals, the help library sources, the manual page, `mkpdf.py`
`etc/`	The sample parameter file, the systemd unit
`tests/`	The tests
`cmake/`	The uninstall script

7.2 Dependencies

Package	Use
StarLet 1.6 or later	Messages, logging, the command line, counted strings, FAO
liburing	The backend
HELP (optional)	The help library and `/HELP`
pandoc and XeLaTeX (optional)	The PDF of the manuals

7.3 Options

Option	Default	Effect
`CMAKE_INSTALL_PREFIX`	`/opt/mscphub`	Where the server is installed
`MH_STRICT`	OFF	Compiler warnings become errors
`BUILD_TESTING`	ON	The tests are built
`MH_SYSTEMD`	ON	The systemd unit is installed (never enabled)
`MH_PDF`	OFF	The manuals are rendered as PDF

7.4 The Checks of the Configuration

CMake refuses a tree in which the `__REV__` of a module does not follow its `__IDENT__` (Section 5.4). The version of the product is read from `src/mscphub.c`.

7.5 The PDF of the Manuals

With `-DMH_PDF=ON`, `doc/mkpdf.py` renders every manual through pandoc and XeLaTeX into a PDF beside its source. The rendered manuals are kept in the repository with their sources, so that the reader does not need TeX to read them.

7.6 The Kit

`cmake --build build --target dist` makes `MSCPHUB-`*ident*`.tar.gz` by `git archive HEAD`: the committed sources, the manuals and the tests, and nothing of a build tree. What is not committed is not in the kit.

Chapter 8. Testing

8.1 What Is There

Test	What it guards
`wire`	The frames the server builds, octet by octet, against reference frames of OpenVMS V8.3 (`tests/t_frames.h`); the channel handshake
`vc`	The virtual circuit and the path: retransmission, duplicates, out of order, an old channel, 30 timeouts, HOSTSHUT, restart
`scs`	SCS and the directory; the MSCP server: STCON, GET UNIT STATUS, ONLINE, AVAILABLE, the checks of a command, credits
`blk`	READ, WRITE, COMPARE and ERASE over the block transfers; the order of the commands; ABORT; a connection that goes while the disk has a command
`bk`	The backend: io_uring, read-modify-write, ranges of sectors, the cache of written sectors, ERASE, group commit, direct I/O
`params`	The checks of the qualifiers
`docs`	Every qualifier and every message is documented
`helplib`	`/HELP` answers out of the library (with the HELP package)

8.2 Rules

- A change that alters what the server sends is not finished without a test that fails before it and passes after it.
- A test is checked by breaking the code it guards and seeing it fail.
- `ctest` passes before anything is installed.

Appendix A. Known Limitations

Limitation	Where it lives
8 disks per server	`/UNIT0` to `/UNIT7` in `s_opts`, `mscphub.c`
One LAN adapter	`mh_lan.c`
64 nodes remembered	`mh_cc.c`, `MANYNODES`
Window of 16 packets per circuit	`mh_tr.c`
No member disk of a shadow set (no forced error)	`mh_mscp.c`
No tapes, no boot of satellites	not implemented
No cluster over IP	not implemented